

=== ПРАВИЛА ОБРАЩЕНИЯ С КОДОМ ===

1. Не переименовывать и не удалять переменные или методы в процессе редактирования.
2. Если выдаваемый тобой код более 3500 символов - разбей его на части.

=== СТРУКТУРА ВЫБРАННЫХ КАТАЛОГОВ ===

[Файл] EliteFolders.cs
[Файл] frmSettings.xaml
[Файл] frmSettings.xaml.cs

=== СОДЕРЖИМОЕ ФАЙЛОВ ===

---EliteFolders.cs---

```
using System;
using System.Diagnostics;
using System.IO;
using System.Linq;
```

```
namespace EDIPSearch.Core;
```

```
public static class EliteFolders
```

```
{
    /// <summary>
    /// Сущность 1: Путь к папке журналов игровых событий (Saved Games).
    /// Всегда фиксирован в профиле пользователя Windows.
    /// </summary>
    public static string SaveGameFolder
    {
        get
        {
            string userProfile = Environment.GetFolderPath(Environment.SpecialFolder.UserProfile);
            return Path.Combine(userProfile, "Saved Games", "Frontier Developments", "Elite Dangerous");
        }
    }
}
```

```
/// <summary>
/// Внутреннее ультимативное ядро для поиска корневой директории установленной игры.
/// </summary>
```

```
private static string GetProductRoot()
{
    try
    {
        // Шаг 1: Если игра запущена, берем путь напрямую из памяти ОС
        var gameProcess = Process.GetProcessesByName("EliteDangerous64").FirstOrDefault();
        if (gameProcess != null)
        {
            string processPath = gameProcess.MainModule?.FileName;
            if (!string.IsNullOrEmpty(processPath))
            {
                string processDir = Path.GetDirectoryName(processPath);
                if (!string.IsNullOrEmpty(processDir) && Directory.Exists(processDir))
                    return processDir;
            }
        }
    }
}
```

```
// Шаг 2: Подстраховка через стандартный реестр Windows (для автономных установщиков)
```

```
using (var key =
Microsoft.Win32.Registry.LocalMachine.OpenSubKey(@"SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\
Dangerous_is1"))
{
    if (key != null)
    {
        string installLoc = key.GetValue("InstallLocation") as string;
        if (!string.IsNullOrEmpty(installLoc))
        {
            string odysseyPath = Path.Combine(installLoc, "Products", "elite-dangerous-odyssey-64");
            if (Directory.Exists(odysseyPath)) return odysseyPath;
        }
    }
}
```

```
string horizonsPath = Path.Combine(installLoc, "Products", "FORC-FDEV-D-1010");
if (Directory.Exists(horizonsPath)) return horizonsPath;
}
}
}
```

```

catch { }
return string.Empty;
}
/// <summary>
/// Сущность 2: Путь к папке конфигурационных XML-файлов (где лежат AppConfig.xml и
AppConfigLocal.xml).
/// Совпадает с корневой директорией продукта.
/// </summary>
public static string ConfigFolder
{
    get
    {
        return GetProductRoot();
    }
}

/// <summary>
/// Сущность 3: Путь к папке сетевых логов игры (Logs).
/// Находится внутри корневой директории установленного продукта.
/// </summary>
public static string NetLogsFolder
{
    get
    {
        string root = GetProductRoot();
        if (string.IsNullOrEmpty(root)) return string.Empty;

        string logsPath = Path.Combine(root, "Logs");

        // Гарантируем физическое существование папки Logs внутри игры
        if (!Directory.Exists(logsPath))
        {
            try { Directory.CreateDirectory(logsPath); } catch { }
        }

        return logsPath;
    }
}

```

```

---frmSettings.xaml---
<Window x:Class="EDIPSearch.frmSettings"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
xmlns:local="clr-namespace:EDIPSearch"
mc:Ignorable="d"
Title="Бортовой компьютер: Настройки конфигурации" Height="450" Width="800"
Background="#11161B" Foreground="#E0E6ED"
Icon="pack://application:,,,/app_icon.ico"
WindowStyle="None"
AllowsTransparency="True">

```

```

<Window.Resources>
<ResourceDictionary>
<ResourceDictionary.MergedDictionaries>
<ResourceDictionary Source="EliteTheme.xaml" />
</ResourceDictionary.MergedDictionaries>
</ResourceDictionary>
</Window.Resources>

```

```

<Border Background="#11161B" BorderBrush="#FF7100" BorderThickness="1">
<Grid>
<Grid.RowDefinitions>
<RowDefinition Height="30"/>
<!-- Row 0: Заголовок -->
<RowDefinition Height="*/>
<!-- Row 1: Контент вкладок -->
</Grid.RowDefinitions>

```

```

<!-- ФИРМЕННЫЙ ЗАГОЛОВОК ОКНА -->
<Grid Grid.Row="0" Background="#252F38" MouseLeftButtonDown="TitleBar_MouseLeftButtonDown">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*" />
<ColumnDefinition Width="Auto" />
</Grid.ColumnDefinitions>
<TextBlock Text="БОРТОВОЙ КОМПЬЮТЕР: НАСТРОЙКИ КОНФИГУРАЦИИ"
Foreground="#FF7100" FontWeight="Bold" VerticalAlignment="Center" Margin="15,0,0,0" FontSize="11"/>
<Button Content="r" FontFamily="Marlett" Width="35" Height="30" Grid.Column="1"
Click="BtnClose_Click" Style="{StaticResource EliteButton}" BorderThickness="0"
Background="Transparent"/>
</Grid>

<!-- ОСНОВНОЙ КОНТЕНТ -->
<DockPanel Grid.Row="1" Margin="10">
<!-- НИЖНЯЯ ПАНЕЛЬ С КНОПКАМИ -->
<Border DockPanel.Dock="Bottom" BorderBrush="#33FF7100" BorderThickness="0,1,0,0"
Padding="0,10,0,0" Margin="0,10,0,0">
<Grid>
<Button x:Name="btnCancel" Style="{StaticResource EliteButton}" Content="ОТМЕНА"
HorizontalAlignment="Left" Width="100" Height="28"/>
<Button x:Name="btnOK" Style="{StaticResource EliteButton}" Content="СОХРАНИТЬ"
HorizontalAlignment="Right" Width="100" Height="28"/>
</Grid>
</Border>

<!-- ВКЛАДКИ НАСТРОЕК -->
<TabControl>
<TabItem Header="Основные настройки">
<Grid Margin="15">
<StackPanel>
<!-- Настройка 1: Авто-отслеживание -->
<CheckBox x:Name="cbListenLog"
Content="Отслеживать активный лог-файл сетевых соединений в реальном времени"
Margin="0,0,0,20"/>

<!-- НАША НОВАЯ НАСТРОЙКА: Удаление файлов -->
<CheckBox x:Name="cbDeleteProcessedLogs"
Content="Принудительно удалять файлы логов после завершения анализа истории"
Foreground="#FFA000"
ToolTip="Автоматически стирает старые обработанные файлы netLog, исключая файлы с текущей датой."
Margin="0,0,0,10"/>

<TextBlock Text="* Внимание: Система безопасности оставляет нетронутыми логи за сегодняшние сутки,
чтобы не прерывать запись активной сессии."
Foreground="#55E0E6ED" FontSize="10" TextWrapping="Wrap" Margin="20,0,0,0"/>
</StackPanel>
</Grid>
</TabItem>

<!-- ВКЛАДКА 2: ФИЛЬТРЫ -->
<TabItem Header="Фильтры" ToolTip="Пул исключений IP-адресов">
<DockPanel Margin="5">
<!-- Тулбар для добавления -->
<ToolBar DockPanel.Dock="Top" Margin="0,0,0,10">
<Label Content="Добавить IP/Подсеть:" VerticalAlignment="Center" Foreground="#FF7100"
Margin="0,0,5,0"/>
<TextBox x:Name="tbAddress" Width="180" Height="24" VerticalContentAlignment="Center"/>

<!-- Плюс и Минус теперь ровные центрированные кнопки -->
<WrapPanel Grid.Row="1" Orientation="Horizontal" HorizontalAlignment="Right" Margin="0,10,0,0">
<Button x:Name="btnFilterAdd" Content="ДОБАВИТЬ" Width="100" Height="26" Margin="0,0,10,0"
Style="{StaticResource EliteButton}" Click="BtnFilterAdd_Click"/>
<Button x:Name="btnFilterRemove" Content="УДАЛИТЬ" Width="100" Height="26" Style="{StaticResource
EliteButton}" Click="BtnFilterRemove_Click"/>
</WrapPanel>
</ToolBar>

<!-- ТАБЛИЦА ФИЛЬТРОВ: Отредерена через GridView колонок -->
<ListView x:Name="lvFilters" Background="Transparent" BorderThickness="0" Foreground="#E0E6ED">
<!-- Умный стиль строк (остаётся без изменений, твоя раскраска комментариев) -->

```

```

<ListView.ItemContainerStyle>
<Style TargetType="ListViewItem" BasedOn="{StaticResource {x:Type ListViewItem}}">
<Style.Triggers>
<DataTrigger Binding="{Binding IsComment}" Value="True">
<Setter Property="Foreground" Value="#55FF7100"/>
<Setter Property="FontStyle" Value="Italic"/>
<Setter Property="Focusable" Value="False"/>
</DataTrigger>
</Style.Triggers>
</Style>
</ListView.ItemContainerStyle>

<ListView.View>
<GridView>
<GridView.ColumnHeaderContainerStyle>
<Style TargetType="GridViewColumnHeader">
<Setter Property="Background" Value="#1B2228"/>
<Setter Property="Foreground" Value="#FF7100"/>
<Setter Property="FontWeight" Value="Bold"/>
<Setter Property="BorderBrush" Value="#FF7100"/>
<Setter Property="BorderThickness" Value="0,0,0,1"/>
<Setter Property="Padding" Value="12,6"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="GridViewColumnHeader">
<Border x:Name="HeaderBorder" Background="{TemplateBinding Background}"
BorderBrush="{TemplateBinding BorderBrush}" BorderThickness="{TemplateBinding BorderThickness}"
Padding="{TemplateBinding Padding}">
<!-- ИСПРАВЛЕНО: Центрируем заголовки для Маски и Размера, если это указано в колонке -->
<ContentPresenter HorizontalAlignment="Center" VerticalAlignment="Center"/>
</Border>
<ControlTemplate.Triggers>
<Trigger Property="IsMouseOver" Value="True">
<Setter TargetName="HeaderBorder" Property="Background" Value="#25FF7100"/>
<Setter Property="Foreground" Value="#FFA000"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>
</GridView.ColumnHeaderContainerStyle>

<!-- Колонки таблицы -->
<!-- 1. IP адрес (выравнивание по левому краю) -->
<GridViewColumn Header="IP АДРЕС / ДИАПАЗОН ИСКЛЮЧЕНИЯ" DisplayMemberBinding="{Binding IP}"
Width="430"/>

<!-- 2. Маска (убрали ошибочный параметр) -->
<GridViewColumn Header="МАСКА" Width="140">
<GridViewColumn.CellTemplate>
<DataTemplate>
<TextBlock Text="{Binding Mask}" TextAlignment="Center" Width="130" Foreground="{Binding
RelativeSource={RelativeSource AncestorType=ListViewItem}, Path=Foreground}"/>
</DataTemplate>
</GridViewColumn.CellTemplate>
</GridViewColumn>

<!-- 3. Размер пула (убрали ошибочный параметр) -->
<GridViewColumn Header="РАЗМЕР ПУЛА" Width="140">
<GridViewColumn.CellTemplate>
<DataTemplate>
<TextBlock Text="{Binding Size}" TextAlignment="Center" Width="130" Foreground="{Binding
RelativeSource={RelativeSource AncestorType=ListViewItem}, Path=Foreground}"/>
</DataTemplate>
</GridViewColumn.CellTemplate>
</GridViewColumn>
</GridView>
</ListView.View>
</ListView>
</DockPanel>
</TabItem>

<!-- ВКЛАДКА 3: ИНТЕГРАЦИЯ -->

```

```

<TabItem Header="Интеграция" ToolTip="Связь с маршрутизатором">
<Grid Margin="15">
<TextBlock Text="Настройки интеграции REST API управляются через отдельный модуль на главном
экране."
Foreground="#88E0E6ED" HorizontalAlignment="Center" VerticalAlignment="Center" TextWrapping="Wrap"
TextAlignment="Center"/>
</Grid>
</TabItem>
</TabControl>
</DockPanel>
</Grid>
</Border>
</Window>

```

```

---frmSettings.xaml.cs---

```

```

using ipinpool;
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Input;
using EDIPSearch.Core;

```

```

namespace EDIPSearch
{
public partial class frmSettings : Window
{
private List<FilterRowItem> _uiRows = new List<FilterRowItem>();
private wiseIPList _filterListForEngine = new wiseIPList();

```

```

public frmSettings()
{
InitializeComponent();

```

```

// ВОЗВРАЩЕНО: Оригинальные названия кнопок и методов клика с большой буквы
btnOK.Click += BtnOK_Click;
btnCancel.Click += BtnCancel_Click;
btnFilterAdd.Click += BtnFilterAdd_Click;
btnFilterRemove.Click += BtnFilterRemove_Click;

```

```

this.Loaded += FrmSettings_Loaded;
}

```

```

private void FrmSettings_Loaded(object sender, RoutedEventArgs e)
{
// Подвязываем параметры к чекбоксам на основе Properties.Settings
cbListenLog.IsChecked = Properties.Settings.Default.AutoMonitoringEnabled;

// Наша новая настройка удаления логов
cbDeleteProcessedLogs.IsChecked = Properties.Settings.Default.DeleteLogsAfterParse;

```

```

LoadFiltersFile();
}

```

```

private void LoadFiltersFile()
{
_uiRows.Clear();

```

```

string workingDir = Properties.Settings.Default.DataFolder;
if (string.IsNullOrEmpty(workingDir))
{
workingDir = Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments),
"EDIPSearch");
}

```

```

string filePath = Path.Combine(workingDir, "filters.txt");
if (!File.Exists(filePath)) return;

try
{
string[] lines = File.ReadAllLines(filePath);
foreach (string line in lines)
{
string trimmed = line.Trim();
if (string.IsNullOrEmpty(trimmed)) continue;

if (trimmed.StartsWith("#"))
{
_uiRows.Add(new FilterRowItem { IsComment = true, RawText = line });
continue;
}

if (trimmed.Contains("#"))
{
int hashIndex = line.IndexOf('#');
string ipPart = line.Substring(0, hashIndex).Trim();
string commentPart = line.Substring(hashIndex).Trim();

// ВОЗВРАЩЕНО: Оригинальное имя метода ValidateAndNormalizeIP
if (ValidateAndNormalizeIP(ipPart, out string normalizedIp, out IPclass ipObj))
{
_uiRows.Add(new FilterRowItem { IsComment = true, RawText = commentPart });
_uiRows.Add(new FilterRowItem { IsComment = false, RawText = normalizedIp, IpObject = ipObj });
}
continue;
}

// ВОЗВРАЩЕНО: Оригинальное имя метода ValidateAndNormalizeIP
if (ValidateAndNormalizeIP(trimmed, out string cleanIp, out IPclass pureIpObj))
{
_uiRows.Add(new FilterRowItem { IsComment = false, RawText = cleanIp, IpObject = pureIpObj });
}
}

// ВОЗВРАЩЕНО: Оригинальное имя метода RefreshGrid
RefreshGrid();
}
catch (Exception ex)
{
MessageBox.Show($"Ошибка чтения: {ex.Message}");
}
}

private void BtnOK_Click(object sender, RoutedEventArgs e)
{
try
{
string targetFolder = Properties.Settings.Default.DataFolder;
if (string.IsNullOrEmpty(targetFolder))
{
targetFolder = Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments),
"EDIPSearch");
}

if (!Directory.Exists(targetFolder)) Directory.CreateDirectory(targetFolder);
string filePath = Path.Combine(targetFolder, "filters.txt");

using (StreamWriter writer = new StreamWriter(filePath, false, Encoding.UTF8))
{
foreach (var row in _uiRows)
{
if (row.IsComment) writer.WriteLine(row.RawText);
else writer.WriteLine(row.IpObject?.ToString() ?? row.RawText);
}
}
}
}

```

```

_filterListForEngine = new wiseIPList();
foreach (var row in _uiRows)
{
    if (!row.IsComment && row.IpObject != null) _filterListForEngine.AddAddress(row.IpObject);
}

// Сохраняем системные настройки
Properties.Settings.Default.AutoMonitoringEnabled = cbListenLog.IsChecked ?? false;

// Фиксируем нашу новую настройку
Properties.Settings.Default.DeleteLogsAfterParse = cbDeleteProcessedLogs.IsChecked ?? false;
Properties.Settings.Default.Save();

this.DialogResult = true;
this.Close();
}
catch (Exception ex) { MessageBox.Show($"Ошибка сохранения параметров: {ex.Message}"); }
}
// Добавьте внутрь класса frmSettings

/// <summary>
/// Обновляет отображение списка фильтров в элементе lvFilters на форме
/// </summary>
private void RefreshGrid()
{
    try
    {
        // Привязываем наш внутренний список строк к ListView на форме
        lvFilters.ItemsSource = null;
        lvFilters.ItemsSource = _uiRows;
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка обновления таблицы: {ex.Message}");
    }
}

/// <summary>
/// Обработчик события клика по кнопке добавления нового фильтра
/// </summary>
private void BtnFilterAdd_Click(object sender, RoutedEventArgs e)
{
    // Открываем встроенный оверлей или диалоговое окно ввода строки фильтра
    // (Используем оригинальную логику вашего приложения для добавления строки)
    string input = Microsoft.VisualBasic.Interaction.InputBox(
        "Введите IP-адрес или подсеть для добавления в белый список исключений:",
        "Добавление фильтра", "");

    if (string.IsNullOrEmpty(input)) return;

    if (ValidateAndNormalizeIP(input, out string normalizedIp, out IPclass ipObj))
    {
        // Проверяем, чтобы не было дубликатов в текущем UI-списке
        if (!_uiRows.Any(x => !x.IsComment && x.RawText.Equals(normalizedIp,
            StringComparison.OrdinalIgnoreCase)))
        {
            _uiRows.Add(new FilterRowItem { IsComment = false, RawText = normalizedIp, IpObject = ipObj });
            RefreshGrid();
        }
    }
    else
    {
        MessageBox.Show("Неверный формат IP-адреса или подсети!", "Ошибка валидации", MessageBoxButton.OK,
            MessageBoxImage.Warning);
    }
}

/// <summary>
/// Обработчик события клика по кнопке удаления выбранного фильтра из списка
/// </summary>
private void BtnFilterRemove_Click(object sender, RoutedEventArgs e)

```

```

{
// Получаем выбранный элемент из списка lvFilters
var selectedItem = lvFilters.SelectedItem as FilterRowItem;
if (selectedItem == null) return;

// Удаляем строку из UI-коллекции
_uiRows.Remove(selectedItem);
RefreshGrid();
}
// Добавьте внутрь класса frmSettings в файле frmSettings.xaml.cs

/// <summary>
/// Оригинальный метод валидации и нормализации строк для белого списка фильтров.
/// </summary>
private bool ValidateAndNormalizeIP(string input, out string normalizedIp, out IPclass ipObj)
{
normalizedIp = input.Trim();
ipObj = null;

if (string.IsNullOrEmpty(normalizedIp)) return false;

// Если маска подсети не указана явно, подставляем /32
if (!normalizedIp.Contains("/"))
{
normalizedIp += "/32";
}

try
{
string[] parts = normalizedIp.Split('/');
if (parts.Length != 2) return false;

// Валидация IP-адреса через стандартный сетевой сокет
if (!System.Net.IPAddress.TryParse(parts[0], out System.Net.IPAddress ip) ||
ip.AddressFamily != System.Net.Sockets.AddressFamily.InterNetwork)
{
return false;
}

// Валидация числового диапазона маски подсети
if (!int.TryParse(parts[1], out int mask) || mask < 0 || mask > 32)
{
return false;
}

// Парсим строку методом ядра в объект IPclass
ipObj = IPclass.Parse(normalizedIp);
return ipObj != null;
}
catch
{
return false;
}
}

private void BtnCancel_Click(object sender, RoutedEventArgs e)
{
this.DialogResult = false;
this.Close();
}

private void TitleBar_MouseLeftButtonDown(object sender, MouseButtonEventArgs e)
{
if (e.ChangedButton == MouseButton.Left) this.DragMove();
}

private void BtnClose_Click(object sender, RoutedEventArgs e)
{
this.DialogResult = false;
this.Close();
}

```

```

}

// Остальные ваши оригинальные методы RefreshGrid(), TbFilterAdd_Click(),
// TbFilterRemove_Click() и ValidateAndNormalizeIp() остаются внутри класса без изменений...
}
}

public class FilterRowItem
{
public bool IsComment { get; set; }
public string RawText { get; set; } = string.Empty;

public string IP => IsComment ? RawText : IpObject?.IP ?? string.Empty;
public string Mask => IsComment ? string.Empty : (IpObject?.PoolSize.ToString() ?? string.Empty);
public string Size => IsComment ? string.Empty : CalculateSize(IpObject?.PoolSize ?? 32);

public IPclass? IpObject { get; set; }

private string CalculateSize(int poolSize)
{
long usableAddresses = 0;

if (poolSize == 32) usableAddresses = 1;
else if (poolSize == 31) usableAddresses = 2;
else
{
long total = (long)Math.Pow(2, 32 - poolSize);
usableAddresses = total - 2;
}

// ИСПРАВЛЕНО: Жестко задаем пробел в качестве разделителя тысяч
var nfi = new System.Globalization.NumberFormatInfo
{
NumberGroupSeparator = " ", // Разделитель - пробел
NumberDecimalDigits = 0 // Отключаем копейки/дробную часть
};

// Форматируем число с использованием нашей маски
return usableAddresses.ToString("N", nfi);
}

}

```